hard links, though, you should first start with the following review.

10.6 Review of hard links

We usually think of a file's name as being the file itself, but really the name is a hard link. A given file can have more than one hard link to itself – for example, a directory has at least two hard links: the directory name and . (for when you're inside it). It also has one hard link from each of its sub-directories (the .. file inside each one). If you have the stat utility installed on your machine, you can find out how many hard links a file has (along with a bunch of other information) with the command:

```
stat filename
```

Hard links aren't just for directories – you can create more than one link to a regular file too. For example, if you have the file a, you can make a link called b:

```
ln a b
```

Now, a and b are two names for the same file, as you can verify by seeing that they reside at the same inode (the inode number will be different on your machine):

```
ls -i a
  232177 a
ls -i b
  232177 b
```

So `ln a b` is roughly equivalent to `cp a b`, but there are several important differences:

1. The contents of the file are only stored once, so you don't use twice the space.
2. If you change a, you're changing b, and vice-versa.
3. If you change the permissions or ownership of a, you're changing those of b as well, and vice-versa.
4. If you overwrite a by copying a third file over it, you will also overwrite b, unless you tell cp to unlink before overwriting. You do this by running cp with the `--remove-destination` flag. Notice that `rsync` always unlinks before overwriting. Note, added 2002.Apr.10: the previous statement applies to changes in the file contents only, not permissions or ownership.

But this raises an interesting question. What happens if you rm one of the links? The answer is that rm is a bit of a misnomer; it doesn't really remove a file, it just removes that one link to it. A file's contents aren't truly removed until the number of links to it reaches zero. In a moment, we're going to make use of that fact, but first, here's a word about cp.

10.7 Using cp -al

In the previous section, it was mentioned that hard-linking a file is similar